Instead, what modern virtualization solutions offer is something known as **live migration**. In other words, they move the virtual machine while it is still operational. For instance, they employ techniques like **pre-copy memory migration**. This means that they copy memory pages while the machine is still serving requests. Most memory pages are not written much, so copying them over is safe. Remember, the virtual machine is still running, so a page may be modified after it has already been copied. When memory pages are modified, we have to make sure that the latest version is copied to the destination, so we mark them as dirty. They will be recopied later. When most memory pages have been copied, we are left with a small number of dirty pages. We now pause very briefly to copy the remaining pages and resume the virtual machine at the new location. While there is still a pause, it is so brief that applications typically are not affected. When the downtime is not noticeable, it is known as a **seamless live migration**.

7.9.3 Checkpointing

Decoupling of virtual machine and physical hardware has additional advantages. In particular, we mentioned that we can pause a machine. This in itself is useful. If the state of the paused machine (e.g., CPU state, memory pages, and storage state) is stored on disk, we have a snapshot of a running machine. If the software makes a royal mess of the still-running virtual machine, it is possible to just roll back to the snapshot and continue as if nothing happened.

The most straightforward way to make a snapshot is to copy everything, including the full file system. However, copying a multiterabyte disk may take a while, even if it is a fast disk. And again, we do not want to pause for long while we are doing it. The solution is to use **copy on write** solutions, so that data are copied only when absolutely necessary.

Snapshotting works quite well, but there are issues. What to do if a machine is interacting with a remote computer? We can snapshot the system and bring it up again at a later stage, but the communicating party may be long gone. Clearly, this is a problem that cannot be solved.

7.10 OS-LEVEL VIRTUALIZATION

So far, we studied hypervisor-based virtualization, but in the beginning of this chapter we mentioned that there is this thing called OS-level virtualization also. Instead of presenting the illusion of a machine, the idea is to create isolated user space environments, also known as a **containers** or **jails**, as mentioned earlier.

As much as possible, each container and all the processes in it are isolated from the other containers and the rest of the system. For instance, they may all have their own file system "name space": a subtree that starts from a root that the administrator created with the chroot system call. A process in this name space

cannot access other name spaces (subtrees). However, having your own root directory is not enough. For proper isolation, containers also need separate name spaces for process identifiers, user identifiers, network interfaces (and the associated IP addresses), IPC endpoints, etc. Furthermore, isolation of (and limits on) memory and CPU usage would also be nice. Perhaps you can think of a few other resources that should be restricted?

We have already seen that limiting access to a particular file system name space using a system call like something like chroot is relatively simple: the operating system remembers that for this group of processes all file operations are relative to the new root. If one of the processes opens a file in /home/hjb/, the operating system knows that this is relative to the new root. We can apply similar tricks to the other name spaces. For instance, when a process opens all network interfaces on the system, the operating makes sure to open only the network interface(s) assigned to the group/container. Similarly, process and user identifiers can be virtualized, so that when a process sends a signal to the process with process identifier 6293, the operating system translates that number to the "real" process identifier. All of this is straightforward. However, how does one partition resources such as memory and CPU usage?

In general, we need a way to track resource usage for groups of processes for a wide variety of resources. Different operating systems have different solutions. One of the better known ones, the Linux **cgroups** (control groups) feature, we will use for illustration purposes. It allows administrators to organize processes in sets known as cgroups, and to monitor and limit a cgroup's usage of various kinds of resources. Cgroups are flexible in that they do not prescribe in advance the exact resources to track. Thus, any resource that can be tracked and restricted can be added. By attaching a resource controller (sometimes referred to as a "subsystem") for a particular resource to a cgroup, it will monitor and/or limit the corresponding resource access for all processes that are a member of that cgroup.

It is possible to limit the usage of one set of resources (such as memory, CPU, and the bandwidth for block I/O) for process P_1 and another set (for instance, only block I/O bandwidth) for process P_2. To do so, we first create two cgroups $C_{CPU+Mem}$ and C_{Blkio}. We then attach the CPU and memory controller to the first cgroup, and the block I/O controller to the second. Finally, we make P_1 a member of both $C_{CPU+Mem}$ and C_{Blkio}, and P_2 a member of only C_{Blkio}.

This in itself is still not enough. Often, we do not want to control the CPU usage of P_1 and P_2 *together,* but rather isolate the CPU usage of P_1 and P_2 *individually* also. As we shall see, cgroups elegantly solve this problem.

It is important to realize that resource control is often possible at different levels of granularity. Take the CPU. At a fine granularity, we can divide the CPU time on a core among processes dynamically using scheduling. We have discussed scheduling at length in Chap. 2. At a much coarser granularity, we can simply divide the cores of a computer, restricting the processes in a cgroup to, say, 4 of the 16 cores. Whatever they do, their processes will not run on the other 12 cores.

While fine-grained CPU control can also be used, core partitioning is actually very popular in practice. The mechanism goes back to the first years of this millennium.

Back in 2004, programmers at Bull SA (the French company that distributed Multics from 1975 until 2000) came up with the notion of **cpusets**. Cpusets allow administrators to associate specific CPUs or cores and subsets of memory to a group of processes. Since then, cpusets have been extended and modified by different programmers from different organizations, among them Paul Menage at Google who also played a leading role in the development of cgroups. This is no coincidence: cpusets match the controller model of cgroups to a t, allowing them to limit a cgroup's usage of CPU and memory at a coarse granularity. In particular, cpusets allow one to assign to a cgroup a set of CPUs and memory nodes—where a memory node simply refers to a node that contains memory, for instance in a NUMA system. Thus, administrators can specify that this cgroup may use these CPU cores and that all its memory will be allocated from the memory at these nodes only. Coarse, but effective!

Moreover, cpusets are hierarchical. In other words, it is possible to subpartition the resources in a parent cpuset into child cpusets. Thus, the root cpuset contains all CPUs and all memory nodes, all level-1 cpusets are subpartitions of its resources, all level-2 cpusets are subpartitions of their level-1 cpusets, and so on. Cgroups are similarly hierarchical. Given the example above, we can create two child cgroups in the parent cgroup $C_{CPU+Mem}$. By attaching different subpartitions of the cpuset with each child cgroup, administrators can ensure that different groups of process keep out of each other's hair.

Using concepts such as cgroups, cpusets, and name spaces, OS-level virtualization allows the creation of isolated containers without resorting to hypervisors or hardware virtualization. These containers have been around for many years, but they really started taking off after the introduction of convenient platforms such as Docker, Kubernetes, and Microsoft Azure Container Registry that help administrators to build, deploy, and manage them.

Compared to hypervisor-based virtual machines, containers are generally more lightweight: faster to start and more efficient in resource consumption. They have other advantages also. For instance, system administration is easier if we need to maintain only a single operating system rather than a separate operating system per virtual machine.

However, there are downsides also. First, you cannot run multiple operating systems on the same machine. If you want to run Windows and UNIX at the same time, containers will not do you much good. Second, while the isolation is pretty good, it is by no means absolute, as the different containers still share the same operating system and may interfere with each other there. If the operating system has static limits on certain resources (such as the number of open files) and one container uses (almost) all of them, the other containers will be in trouble. Similarly, a single vulnerability in the operating system endangers all the containers. In comparison, the isolation offered by hypervisors is considerably stronger. Also,

some researchers argue that hypervisor-based virtualization need not be more heavyweight than containers at all, as long as you reduce the virtual machines to a unikernel (Manco et al., 2017).

7.11 CASE STUDY: VMWARE

Since 1999, VMware, Inc. has been the leading commercial provider of hypervisor-based virtualization solutions with products for desktops, servers, the cloud, and now even on cell phones. It provides not only hypervisors but also the software that manages virtual machines on a large scale.

We will start this case study with a brief history of how the company got started. We will then describe VMware Workstation, a type 2 hypervisor and the company's first product, the challenges in its design and the key elements of the solution. We then describe the evolution of VMware Workstation over the years. We conclude with a description of ESX Server, VMware's type 1 hypervisor.

7.11.1 The Early History of VMware

Although the idea of using virtual machines was popular in the 1960s and 1970s in both the computing industry and academic research, interest in virtualization was totally lost after the 1980s and the rise of the personal computer industry. Only IBM's mainframe division still cared about virtualization. Indeed, the computer architectures designed at the time, and in particular Intel's x86 architecture, did not provide architectural support for virtualization (i.e., they failed the Popek/Goldberg criteria). This is extremely unfortunate, since the 386 CPU, a complete redesign of the 286, was done a decade after the Popek-Goldberg paper, and the designers should have known better.

In 1997, at Stanford, three of the future founders of VMware had built a prototype hypervisor called Disco (Bugnion et al., 1997), with the goal of running commodity operating systems (in particular UNIX) on a very large scale multiprocessor then being developed at Stanford: the FLASH machine. During that project, the authors realized that using virtual machines could solve, simply and elegantly, a number of hard system software problems: rather than trying to solve these problems within existing operating systems, one could innovate in a layer **below** existing operating systems. The key observation of Disco was that, while the high complexity of modern operating systems made innovation difficult, the relative simplicity of a virtual machine monitor and its position in the software stack provided a powerful foothold to address limitations of operating systems. Although Disco was aimed at very large servers, and designed for the MIPS architecture, the authors realized that the same approach could equally apply, and be commercially relevant, for the x86 marketplace.